

H1 SpringBoot源码分析

H2 二、启动原理

```
SpringApplication springApplication = new SpringApplication(主启动类.class);
```

1.启动Spring应用

H3

```
/**
 * Create a new {@link SpringApplication} instance. The application context
will load
 * beans from the specified primary sources (see {@link SpringApplication
class-level}
 * documentation for details). The instance can be customized before calling
 * {@link #run(String...)}.
 * @param primarySources the primary bean sources
 * @see #run(Class, String[])
 * @see #SpringApplication(ResourceLoader, Class...)
 * @see #setSources(Set)
 */
public SpringApplication(Class<?>... primarySources) {
    this(null, primarySources);
}
```

```
/**
 * Run the Spring application, creating and refreshing a new
 * {@link ApplicationContext}.
 * @param args the application arguments (usually passed from a Java main
method)
 * @return a running {@link ApplicationContext}
 */
public ConfigurableApplicationContext run(String... args) {
    // 创建启动跟踪器，用于记录应用启动性能指标
    Startup startup = Startup.create();
    // 如果配置了注册关闭钩子，启用关闭钩子添加功能
```

//“注册关闭钩子”是指向JVM注册一个在应用程序正常关闭时会被调用的特殊方法。这段代码的作用是：如果SpringApplication被配置为注册关闭钩子(registerShutdownHook=true)，它会启用关闭钩子的添加功能，确保当JVM关闭时(如按Ctrl+C或系统关闭)，Spring容器能够优雅地关闭，执行必要的清理工作，比如：调用Bean的销毁方法/关闭数据库连接/释放文件句柄/完成正在处理的请求/这样可以防止资源泄露，保证应用程序干净地退出。

```
if (this.registerShutdownHook) {
    SpringApplication.shutdownHook.enableShutdownHookAddition();
}
// 创建引导上下文，用于应用启动前的准备工作
//引导上下文是Spring Boot应用启动过程中的“预热”环境，为主应用上下文的创建和初始化做准备工作。引导上下文(Bootstrap Context)是Spring Boot启动过程中的一个初始化阶段的上下文环境，它具有以下特点和作用：
```

//生命周期短暂：它只存在于应用程序启动的早期阶段，在完整的ApplicationContext创建和刷新之前。

//主要用途：提供启动前的准备环境：支持在主应用上下文创建前执行初始化逻辑，存储和管理启动过程中需要的临时资源。作为桥梁：连接应用启动前的初始化阶段和正式运行阶段

//使用场景：注册早期监听器/预加载某些服务/执行必须在主上下文创建前完成的配置/支持启动阶段的进度报告

```
DefaultBootstrapContext bootstrapContext = createBootstrapContext();
// 初始化应用上下文变量
ConfigurableApplicationContext context = null;
// 配置java.awt.headless属性，适用于无显示器环境
configureHeadlessProperty();
// 获取并初始化所有SpringApplicationRunListener实例
SpringApplicationRunListeners listeners = getRunListeners(args);
// 通知所有监听器：应用正在启动
listeners.starting(bootstrapContext, this.mainApplicationClass);
try {
    // 将命令行参数封装为ApplicationArguments对象
    ApplicationArguments applicationArguments = new
DefaultApplicationArguments(args);
    // 准备应用环境，包括配置文件加载、属性源设置等
    ConfigurableEnvironment environment = prepareEnvironment(listeners,
bootstrapContext, applicationArguments);
    // 打印应用启动横幅（Banner）
    Banner printedBanner = printBanner(environment);
    // 创建应用上下文（根据应用类型可能是web或非web上下文）
    context = createApplicationContext();
    // 设置应用启动跟踪器到上下文
    context.setApplicationStartup(this.applicationStartup);
    // 准备上下文，注册BeanPostProcessors，加载配置类等
    prepareContext(bootstrapContext, context, environment, listeners,
applicationArguments, printedBanner);
    // 【核心操作】刷新上下文，实例化所有非懒加载的单例Bean
    //此方法调用会触发Spring容器的刷新过程，其中包括：Bean定义的加载和解析，Bean工厂
的准备和后处理、Bean的实例化、初始化和依赖注入。各种Bean后处理器的应用，单例Bean的预先实例化。
    //这是Spring应用上下文生命周期中最关键的一步，它完成了从Bean定义到实际Bean实例的
转换过程，将配置信息转变为可用的应用组件。在此过程中，Spring会扫描、加载和初始化所有声明的Bean。
    refreshContext(context);
    // 执行上下文刷新后的自定义逻辑
    afterRefresh(context, applicationArguments);
    // 标记应用启动完成
    startup.started();
}
```

```

        // 如果启用了启动信息日志，记录应用已启动的日志
        if (this.logStartupInfo) {
            new
startupInfoLogger(this.mainApplicationClass).logStarted(getApplicationLog(),
startup);
        }
        // 通知所有监听器：应用已启动完成
        listeners.started(context, startup.timeTakenToStarted());
        // 调用所有注册的ApplicationRunner和CommandLineRunner
        callRunners(context, applicationArguments);
    }
    catch (Throwable ex) {
        // 处理启动过程中的异常，可能关闭上下文并通知监听器
        throw handleRunFailure(context, ex, listeners);
    }
    try {
        // 如果上下文正在运行，通知所有监听器：应用已就绪
        if (context.isRunning()) {
            listeners.ready(context, startup.ready());
        }
    }
    catch (Throwable ex) {
        // 处理就绪阶段的异常
        throw handleRunFailure(context, ex, null);
    }
    // 返回已配置并运行的应用上下文
    return context;
}

```

Spring应用启动过程详解

Spring应用的启动过程可以比作一家餐厅从早晨开门到正式营业的全过程，下面用通俗语言详细解释这段代码的执行流程：

1. 准备工作

• ```java

```

Startup startup = Startup.create();
if (this.registerShutdownHook) {
    SpringApplication.shutdownHook.enableShutdownHookAddition();
}

```

• ```

这就像餐厅老板一大早来开门，启动计时器记录开业准备时间，并安排好打烊的计划（注册关闭钩子，确保程序能优雅地关闭）。

2. 搭建临时工作区

• ```java

```

DefaultBootstrapContext bootstrapContext = createBootstrapContext();

```

• ```

相当于厨师们先搭建一个临时工作台，放置一些前期准备工作需要的工具。

3. 基础设置

```
•```java
ConfigurableApplicationContext context = null;
configureHeadlessProperty();
•```
```

为正式的餐厅环境预留位置，并设置一些基本的系统属性，比如告诉系统这是一个不需要显示器的应用。

4. 通知开始准备

```
•```java
SpringApplicationRunListeners listeners = getRunListeners(args);
listeners.starting(bootstrapContext, this.mainApplicationClass);
•```
```

就像餐厅经理召集所有员工开晨会，告诉大家："我们要开始准备开业了！"，让各个部门做好准备。

5. 处理命令参数

```
•```java
ApplicationArguments applicationArguments = new DefaultApplicationArguments(args);
•```
```

这相当于餐厅经理整理当天的特殊安排和注意事项（命令行参数）。

6. 准备环境

```
•```java
ConfigurableEnvironment environment = prepareEnvironment(listeners,
bootstrapContext, applicationArguments);
•```
```

就像调整餐厅的温度、湿度、灯光等环境因素，设置运行环境的各种配置和属性。

7. 展示标识

```
•```java
Banner printedBanner = printBanner(environment);
•```
```

相当于挂出餐厅的招牌和今日特色菜单，在控制台打印Spring的标志。

8. 创建应用上下文

```
•```java
context = createApplicationContext();
context.setApplicationStartup(this.applicationStartup);
•```
```

这就是正式搭建餐厅的营业大厅，这是Spring应用的核心容器，所有的Bean都将在这里被管理。

9. 准备上下文

```
•```java
prepareContext(bootstrapContext, context, environment, listeners,
applicationArguments, printedBanner);
•```
```

相当于布置餐厅，摆放桌椅，准备餐具，让一切就绪。这一步会加载所有的Bean定义，但还没有实例化它们。

10. 刷新上下文（核心步骤）

```
•```java
refreshContext(context);
•```
```

这是整个过程的核心，就像厨师们开始烹饪菜品，服务员准备就位。**Spring**会在这一步实例化所有的**Bean**，注入依赖，并调用各种初始化方法。

11. 刷新后处理

```
• ```java
afterRefresh(context, applicationArguments);
• ```
```

类似于最后的检查，确保所有准备工作都已完成，可能会执行一些自定义的后处理操作。

12. 标记启动完成

```
• ```java
startup.started();
• ```
```

打卡下班，记录从开始到现在所花费的时间。

13. 记录启动信息

```
• ```java
if (this.logStartupInfo) {
    new
StartupInfoLogger(this.mainApplicationClass).logStarted(getApplicationLog(),
startup);
}
• ```
```

在日志中记录餐厅成功开业的消息，包括花了多长时间准备。

14. 通知启动完成

```
• ```java
listeners.started(context, startup.timeTakenToStarted());
• ```
```

向所有部门宣布："我们已经正式开业了！"让所有监听器知道应用已经启动。

15. 运行特定任务

```
• ```java
callRunners(context, applicationArguments);
• ```
```

执行一些需要在应用启动后立即运行的特殊任务，就像餐厅开门后的第一批特殊服务。

16. 处理就绪状态

```
• ```java
if (context.isRunning()) {
    listeners.ready(context, startup.ready());
}
• ```
```

确认餐厅一切运转正常后，向所有人宣布："我们已准备好接待客人了！"

17. 返回结果

```
• ```java
return context;
• ```
```

整个启动过程完成，返回已经准备就绪的应用上下文，就像餐厅完全开业，可以正常接待顾客了。

在整个过程中，**spring**还设置了异常处理机制，就像餐厅有应急预案一样，如果任何环节出现问题，都会有专门的处理方式，确保能够妥善处理故障情况。

2.Bean生命周期与加载优化

SpringBoot bean的生命周期：要求对应的核心源码阐述，并且如何优化bean加载，减少30%无效Bean加载？

H3

H2 一、Bean生命周期核心源码分析

Bean的生命周期主要由Spring容器中的 `BeanFactory` 管理，在SpringBoot中使用的是 `DefaultListableBeanFactory` 实现。以下是Bean完整生命周期的关键阶段及对应源码：

1. Bean定义加载阶段

Bean定义首先通过各种 `BeanDefinitionReader` 实现类被加载到容器中：

H3

```
// ConfigurationClassPostProcessor.java
@Override
public void postProcessBeanDefinitionRegistry(BeansDefinitionRegistry registry) {
    // 处理配置类
    processConfigBeanDefinitions(registry);
}

// 加载@Bean、@Import等注解定义的Bean
private void processConfigBeanDefinitions(BeansDefinitionRegistry registry) {
    List<BeansDefinitionHolder> configCandidates = new ArrayList<>();
    // 查找配置类
    String[] candidateNames = registry.getBeanDefinitionNames();

    // 解析配置类，注册Bean定义
    ConfigurationClassParser parser = new ConfigurationClassParser(
        this.metadataReaderFactory, this.problemReporter, this.environment,
        this.resourceLoader, this.componentScanBeanNameGenerator, registry);
    parser.parse(candidateNames);
}
```

2. Bean实例化阶段

当请求获取Bean时, `AbstractBeanFactory.getBean()` 方法触发Bean的创建流程:

H3

```
// AbstractBeanFactory.java
@Override
public Object getBean(String name) throws BeansException {
    return doGetBean(name, null, null, false);
}

protected <T> T doGetBean(String name, @Nullable Class<T> requiredType,
                           @Nullable Object[] args, boolean typeCheckOnly) {
    // 转换Bean名称
    String beanName = transformedBeanName(name);
    Object bean;

    // 检查单例缓存
    Object sharedInstance = getSingleton(beanName);
    if (sharedInstance != null && args == null) {
        bean = getObjectForBeanInstance(sharedInstance, name, beanName, null);
    }
    else {
        // 创建Bean实例
        if (mbd.isSingleton()) {
            sharedInstance = getSingleton(beanName, () -> {
                try {
                    // 这里调用createBean创建Bean
                    return createBean(beanName, mbd, args);
                }
                catch (BeansException ex) {
                    destroySingleton(beanName);
                    throw ex;
                }
            });
            bean = getObjectForBeanInstance(sharedInstance, name, beanName, mbd);
        }
    }
    return (T) bean;
}
```

3. Bean实例创建与初始化

`AbstractAutowireCapableBeanFactory.createBean()` 方法负责Bean的实际创建:

H3

```
// AbstractAutowireCapableBeanFactory.java
@Override
protected Object createBean(String beanName, RootBeanDefinition mbd, @Nullable
Object[] args) {
    // 解析Bean类
    Class<?> resolvedClass = resolveBeanClass(mbd, beanName);
}
```

```

    try {
        // 给BeanPostProcessors机会返回代理对象
        Object bean = resolveBeforeInstantiation(beanName, mbd);
        if (bean != null) {
            return bean;
        }
    }
    catch (Throwable ex) {
        throw new BeanCreationException(mbd.getResourceDescription(), beanName,
            "BeanPostProcessor before instantiation of bean failed", ex);
    }

    try {
        // 创建Bean实例
        Object beanInstance = doCreateBean(beanName, mbd, args);
        return beanInstance;
    }
    catch (BeanCreationException | ImplicitlyAppearedSingletonException ex) {
        throw ex;
    }
}

protected Object doCreateBean(String beanName, RootBeanDefinition mbd, @Nullable
Object[] args) {
    // 实例化Bean
    BeanWrapper instanceWrapper = createBeanInstance(beanName, mbd, args);
    Object bean = instanceWrapper.getWrappedInstance();

    // 填充属性（依赖注入）
    populateBean(beanName, mbd, instanceWrapper);

    // 初始化Bean
    Object exposedObject = initializeBean(beanName, bean, mbd);
    return exposedObject;
}

```

4. 初始化回调执行

`initializeBean` 方法执行各种初始化回调：

H3

```

// AbstractAutowireCapableBeanFactory.java
protected Object initializeBean(String beanName, Object bean, @Nullable
RootBeanDefinition mbd) {
    // 执行Aware接口方法
    invokeAwareMethods(beanName, bean);

    Object wrappedBean = bean;
    // 执行BeanPostProcessor前置处理
    wrappedBean = applyBeanPostProcessorsBeforeInitialization(wrappedBean,
        beanName);

    // 执行初始化方法

```



```

        invokeInitMethods(beanName, wrappedBean, mbd);

        // 执行BeanPostProcessor后置处理
        wrappedBean = applyBeanPostProcessorsAfterInitialization(wrappedBean,
            beanName);
        return wrappedBean;
    }

```

5. 销毁阶段

当容器关闭时，会调用Bean的销毁方法：

H3

```

// DisposableBeanAdapter.java
@Override
public void destroy() {
    // 执行自定义销毁方法
    if (this.invokeDisposableBean) {
        try {
            // 调用DisposableBean接口的destroy方法
            ((DisposableBean) this.bean).destroy();
        }
        catch (Throwable ex) {
            // 异常处理
        }
    }

    // 执行自定义销毁方法（如@PreDestroy注解的方法）
    if (this.destroyMethod != null) {
        invokeCustomDestroyMethod(this.destroyMethod);
    }
}

```

H2

二、优化Bean加载，减少30%无效Bean加载

1. 懒加载策略应用

H3

```

@Configuration
public class LazyLoadingConfig {

    // 1. 使用@Lazy注解延迟加载单个Bean
    @Bean
    @Lazy
    public ExpensiveService expensiveService() {
        return new ExpensiveServiceImpl();
    }

    // 2. 全局配置懒加载
    @PostConstruct
    public void configLazyLoading() {

```

```

        // 可在启动类中设置
        ConfigurableBeanFactory beanFactory = applicationContext.getBeanFactory();
        beanFactory.setAllowBeanDefinitionOverriding(true);
        beanFactory.setAllowCircularReferences(false);
    }
}

// 3. 在application.properties中配置
// spring.main.lazy-initialization=true

```

2. 条件化Bean注册

H3

```

// 1. 使用@Conditional系列注解
@Configuration
public class ConditionalBeanConfig {

    @Bean
    @ConditionalOnProperty(name = "feature.advanced-reporting.enabled",
        havingValue = "true")
    public AdvancedReportingService advancedReportingService() {
        return new AdvancedReportingServiceImpl();
    }

    @Bean
    @ConditionalOnExpression("${env.type} == 'production' and
        ${metrics.enabled:false}")
    public MetricsCollector productionMetricsCollector() {
        return new ProductionMetricsCollector();
    }

    @Bean
    @ConditionalOnMissingBean(AuditService.class)
    public DefaultAuditService defaultAuditService() {
        return new DefaultAuditService();
    }
}

```

3. 配置分离与按需加载

H3

```

// 1. 使用@Profile注解区分不同环境
@Configuration
@Profile("production")
public class ProductionConfig {
    // 仅在生产环境加载的Bean
}

@Configuration
@Profile("development")
public class DevelopmentConfig {
    // 仅在开发环境加载的Bean
}

```

```

}

// 2. 使用模块化配置
@Configuration
@Import({
    CoreConfig.class, // 核心模块总是加载
    // 按需导入的模块
    @ConditionalOnProperty(name = "module.reporting.enabled", havingValue =
"true")
    ReportingConfig.class,
    @ConditionalOnProperty(name = "module.admin.enabled", havingValue = "true")
    AdminConfig.class
})
public class ModularConfig {
}

```

4. 定制BeanDefinitionRegistryPostProcessor

创建自定义的 `BeanDefinitionRegistryPostProcessor` 来过滤不需要的Bean定义：

H3

```

@Component
public class OptimizedBeanDefinitionRegistryPostProcessor
    implements BeanDefinitionRegistryPostProcessor, EnvironmentAware {

    private Environment environment;

    @Override
    public void setEnvironment(Environment environment) {
        this.environment = environment;
    }

    @Override
    public void postProcessBeanDefinitionRegistry(BeanDefinitionRegistry registry)
        throws BeansException {
        // 获取当前激活的profiles
        String[] activeProfiles = environment.getActiveProfiles();
        Set<String> activeProfilesSet = new HashSet<>
(Arrays.asList(activeProfiles));

        // 获取所有Bean定义名称
        String[] beanNames = registry.getBeanDefinitionNames();
        for (String beanName : beanNames) {
            BeanDefinition beanDefinition = registry.getBeanDefinition(beanName);

            // 检查是否有自定义属性标记为可选Bean
            if (isOptionalBean(beanDefinition) &&
!isRequiredInCurrentContext(beanDefinition, activeProfilesSet)) {
                // 移除不需要的Bean定义
                registry.removeBeanDefinition(beanName);
            }
        }
    }
}

```

```

private boolean isOptionalBean(BeanDefinition beanDefinition) {
    // 实现检查逻辑，例如基于注解或命名约定
    return beanDefinition.getSource() != null &&
        beanDefinition.getSource().toString().contains("optional");
}

private boolean isRequiredInCurrentContext(BeanDefinition beanDefinition,
                                           Set<String> activeProfiles) {

    // 检查Bean是否需要在当前上下文中加载
    // 可以基于各种条件如profile、属性值等
    return true; // 实现具体逻辑
}

@Override
public void postProcessBeanFactory(ConfigurableListableBeanFactory
beanFactory)
    throws BeansException {
    // 不需要实现
}
}

```

5. 基于ASM的编译时优化

创建一个编译时注解处理器，在编译期分析Bean依赖关系：

H3

```

@Component
public class BeanDependencyAnalyzer {

    private final ConfigurableListableBeanFactory beanFactory;

    public BeanDependencyAnalyzer(ConfigurableListableBeanFactory beanFactory) {
        this.beanFactory = beanFactory;
    }

    @PostConstruct
    public void analyze() {
        // 在应用启动时分析Bean依赖关系并输出报告
        Map<String, Set<String>> unusedBeans = findUnusedBeans();
        log.info("发现{}个未被使用的Bean，可考虑条件化加载", unusedBeans.size());

        // 输出优化建议
        for (Map.Entry<String, Set<String>> entry : unusedBeans.entrySet()) {
            log.info("Bean[{}] 未被其他Bean依赖，可考虑使用@Lazy或@Conditional",
                entry.getKey());
        }
    }

    private Map<String, Set<String>> findUnusedBeans() {
        // 实现依赖分析逻辑
        Map<String, Set<String>> dependencyMap = new HashMap<>();
    }
}

```

```

        // 分析所有Bean的依赖关系
        for (String beanName : beanFactory.getBeanDefinitionNames()) {
            BeanDefinition bd = beanFactory.getBeanDefinition(beanName);
            // 分析Bean的依赖关系并构建依赖图
        }

        return dependencyMap;
    }
}

```

6. 实现自定义的BeanFactoryPostProcessor

H3

```

@Component
public class OptimizedBeanFactoryPostProcessor implements BeanFactoryPostProcessor
{

    @Override
    public void postProcessBeanFactory(ConfigurableListableBeanFactory
beanFactory)
        throws BeansException {
        // 获取所有Bean定义
        for (String beanName : beanFactory.getBeanDefinitionNames()) {
            BeanDefinition beanDefinition =
                ((BeanDefinitionRegistry)
beanFactory).getBeanDefinition(beanName);

            // 检查Bean类型
            if (isInfrequentlyUsedBean(beanDefinition)) {
                // 将Bean设置为懒加载
                beanDefinition.setLazyInit(true);
            }
        }
    }

    private boolean isInfrequentlyUsedBean(BeanDefinition beanDefinition) {
        // 实现检测逻辑，例如基于命名约定或特定包
        String className = beanDefinition.getBeanClassName();
        if (className == null) {
            return false;
        }

        // 例如：管理控制台相关的Bean、报表生成器等不常用功能
        return className.contains("admin") ||
            className.contains("report") ||
            className.contains("statistics");
    }
}

```

7. 使用Spring Boot的自动配置优化

H3

```
// 1. 禁用不需要的自动配置
@SpringBootApplication(exclude = {
    DataSourceAutoConfiguration.class,
    HibernateJpaAutoConfiguration.class,
    SecurityAutoConfiguration.class
})
public class OptimizedApplication {
    public static void main(String[] args) {
        SpringApplication.run(OptimizedApplication.class, args);
    }
}

// 2. 在application.properties中配置
//
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

8. 启动性能监控与优化建议

H3

```
@Component
public class BeanInitializationBenchmark implements
    ApplicationListener<ContextRefreshedEvent> {

    private static final Logger log =
        LoggerFactory.getLogger(BeanInitializationBenchmark.class);

    @Override
    public void onApplicationEvent(ContextRefreshedEvent event) {
        ApplicationContext context = event.getApplicationContext();
        ConfigurableListableBeanFactory beanFactory =
            ((ConfigurableApplicationContext) context).getBeanFactory();

        // 收集初始化时间较长的Bean
        Map<String, Long> beanInitTimes = new HashMap<>();

        for (String beanName : beanFactory.getBeanDefinitionNames()) {
            if (!beanFactory.isLazyInit(beanName) &&
                !beanFactory.containsSingleton(beanName)) {
                long startTime = System.nanoTime();
                beanFactory.getBean(beanName);
                long endTime = System.nanoTime();
                long initTime = (endTime - startTime) / 1_000_000; // 转为毫秒

                if (initTime > 100) { // 超过100ms的初始化时间
                    beanInitTimes.put(beanName, initTime);
                }
            }
        }
    }
}
```

```
// 输出优化建议
if (!beanInitTimes.isEmpty()) {
    log.info("以下Bean初始化时间较长，建议使用懒加载或优化初始化过程:");
    beanInitTimes.entrySet().stream()
        .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
        .forEach(entry -> log.info("Bean[{}] 初始化耗时: {}ms",
            entry.getKey(), entry.getValue()));
}
}
```

通过以上优化策略的综合应用，可以有效减少30%